

This note is about using model-generated rollouts to train the reward-conditioned trajectory model through the interval consistency identity. Generated rollouts are never fitted as data: no likelihood term is applied to a generated trajectory or to a model-produced reward label. They only supply prefixes, continuations, and reward queries on which the identity is enforced. All heads remain trainable.

Setup

Let h_i be a trajectory prefix and $x_{i+1:j}$ a continuation, giving h_j . A block x_t includes the modeled actions, states, and termination tokens—not just actions.

Write the three predictions as:

$$\begin{aligned} p_\theta(x \mid h_i) & \text{ NOR trajectory prediction} \\ p_\theta(x \mid h_i, r) & \text{ reward-conditioned trajectory prediction} \\ q_\theta(r \mid h_i) & \text{ NOR reward prediction} \end{aligned}$$

A generated continuation is drawn by requesting a reward c :

$$x_{i+1:j} \sim G_\theta(\cdot \mid h_i, c).$$

Here G_θ is the generation procedure: reward-conditioned actions, NOR dynamics, and learned termination. The request c shapes which states get visited. It is not a label: nothing in training asserts that the generated continuation achieved c , or any other reward.

1. The consistency identity

For a prefix h_i , a continuation $x_{i+1:j}$, and any reward r , Bayes consistency requires

$$p_\theta(x_{i+1:j} \mid h_i, r) \cdot q_\theta(r \mid h_i) = p_\theta(x_{i+1:j} \mid h_i) \cdot q_\theta(r \mid h_j).$$

Its log residual is:

$$\Delta_{i,j}(r) = \log p_\theta(x_{i+1:j} \mid h_i, r) - \log p_\theta(x_{i+1:j} \mid h_i) + \log q_\theta(r \mid h_i) - \log q_\theta(r \mid h_j).$$

The update minimizes:

$$\mathcal{L}_{\text{cons}} = \mathbb{E}_{i < j} [\Delta_{i,j}(r)^2].$$

This gives a relationship rather than a target:

$$\frac{q_\theta(r \mid h_j)}{q_\theta(r \mid h_i)} = \frac{p_\theta(x_{i+1:j} \mid h_i, r)}{p_\theta(x_{i+1:j} \mid h_i)}.$$

The continuation should increase belief in a reward precisely when it is more likely under that reward condition. The identity holds for every r , not only a reward the trajectory achieved; r is a query.

With full gradients, the loss can change both trajectory predictions and both reward predictions. No endpoint is designated the correct teacher. For an isolated block, writing $v = \log p_\theta(x \mid h_i, r)$,

$$\frac{\partial \mathcal{L}_{\text{cons}}}{\partial v} = 2\Delta_{i,j}(r),$$

so the update can favor or disfavor the sampled continuation under its query reward, depending on the other predictions. Nothing in the objective makes an observed token more likely merely because it was observed.

Because the reward-conditioned prediction is normalized over continuations, the identity summed over all continuations of h_i gives the marginal relation

$$q_i(r) = \mathbb{E}_{x \sim p(\cdot \mid h_i)}[q_j(r)].$$

Given the NOR trajectory model and the terminal anchoring of q on recorded outcomes, the identity therefore determines the reward predictions at every prefix and the reward-conditioned model at every prefix–reward pair, including pairs the offline data never contains. Enforcing it where those pairs occur is the purpose of generated rollouts.

2. Query rewards from the reward prediction

Two rewards enter: the request c that shapes generation, and the query r in the residual. Either can use the model’s estimate of which outcomes are plausible at the current prefix. Write

$$q_i(k) = q_\theta(R = r_k \mid h_i), \quad k = 0, \dots, K-1,$$

where the bins are ordered by increasing reward. Read this distribution in NOR mode, before inserting a requested reward. Choose the request once for a generated continuation; the episode reward condition stays fixed as the prefix grows.

A general upward bias has the form

$$\rho_i(k) = \frac{q_i(k) w(k)}{\sum_{\ell=0}^{K-1} q_i(\ell) w(\ell)}, \quad 0 < w(0) \leq \dots \leq w(K-1) < \infty.$$

For example, exponential tilting uses

$$w(k) = \exp(\beta r_k), \quad \beta \geq 0,$$

with representative reward values r_k , rather than treating bin indices as evenly spaced rewards. A reward assigned zero probability by q_i is never requested. Uniform sampling above a recorded outcome has no corresponding dependence on the model’s feasibility estimate. The plan is to increase β over training, so that ambitious queries become common only once the reward predictions and the conditioned policy are accurate enough to support them.

This is a soft preference, not a feasibility guarantee. A softmax reward head generally assigns positive probability to every bin, including impossible ones. An upward bias can amplify erroneous tail mass, and an extremely strong bias can overwhelm the model’s preference for plausible outcomes. Moreover, q_i describes outcomes under the learned NOR trajectory distribution; low probability can mean either an impossible outcome or a possible outcome that this behavior rarely achieves.

Hard conditioning on rewards above the predicted mean is another choice:

$$\mu_i = \sum_{k=0}^{K-1} q_i(k) r_k,$$

$$\rho_i^{\text{tail}}(k) = \frac{q_i(k) \mathbf{1}\{r_k > \mu_i\}}{\sum_{\ell=0}^{K-1} q_i(\ell) \mathbf{1}\{r_\ell > \mu_i\}}.$$

It requires a fallback when the denominator is zero. It also renormalizes the upper tail to total probability one, even if that tail consists almost entirely of a tiny amount of erroneous probability. A mild soft bias avoids that forced concentration.

Any request rule gives the generated-continuation distribution

$$g_\theta(x \mid h_i) = \sum_{k=0}^{K-1} \rho_i(k) G_\theta(x \mid h_i, r_k).$$

These are proposed rules. The current implementation requests uniformly above the recorded offline outcome and queries the residual at the rollout’s own recorded bin; the model-adaptive sampler has not been implemented. The implemented augmentation also fits relabelled rollouts as data, which is outside this plan.

3. Generated rollouts as a proposal distribution

Let $\nu(h_i, x, r)$ be any sampling distribution over prefixes, continuations, and reward queries for which the log factors are defined. The consistency objective is

$$\mathcal{L}_{\text{cons}}(\nu) = \mathbb{E}_{(h_i, x, r) \sim \nu} [\Delta_{i,j}(r)^2].$$

A coherent model has $\Delta_{i,j}(r) = 0$ pointwise. It therefore has zero consistency loss under every such proposal distribution. Changing ν changes which disagreements receive optimization effort, without changing the identity that defines agreement. With incomplete coverage or limited capacity, the selected proposal still matters to the learned solution.

This is the reason to run consistency on deliberately unusual prefixes and high-reward queries: they probe relationships among the model’s beliefs without becoming demonstrations that every observed token should be made more likely. The generated distribution g_θ of section 2 is one such proposal. Its role is to

reach prefixes that the offline data never produces, such as a far start followed by goal-directed steps, and to pair them with the reward queries the test asks about. Squared log residuals measure relative disagreement even for rare events, but rare events must still be sampled or deliberately queried to receive this signal.

A rare reward marginal does not imply an unreliable conditional policy

The goal is reliable high reward when a high reward is requested. It does not require high reward to be common under NOR behavior. For an exactly represented joint distribution with a deterministic trajectory reward, a supported reward condition satisfies

$$\Pr_{x \sim p(\cdot|h_i, r)} \{R(x) = r\} = 1$$

even if $q_i(r)$ is extremely small. Here $R(x)$ describes the ideal joint distribution; no environment reward evaluator is added to training. This is a statement about exact conditional sampling, not a guarantee for the implemented generator that combines conditioned actions with NOR dynamics.

Thus consistency can improve the high-reward conditional by coordinating beliefs already learned from offline data, without making the NOR reward marginal optimistic. In the actual model, reliable reward conditioning requires learning the missing relationships; their mathematical existence is not enough.

Failure modes

Consistency admits unhelpful solutions. If both token modes ignore the reward and the reward prediction is constant along a trajectory, every interval residual is zero. It can also make several incorrect predictions agree. The objective alone does not distinguish a real relationship from a mutually supported model error; the offline likelihood terms and the terminal anchoring of q are what select among coherent joints.

The training objective is

$$\mathcal{L} = \mathcal{L}_{\text{offline}} + \beta \mathbb{E}_{\nu}[\mathcal{L}_{\text{cons}}],$$

with $\mathcal{L}_{\text{offline}}$ the supervised objective on recorded trajectories and their recorded outcomes, and ν ranging over offline rows, generated rollouts, or a mixture. Compare proposals at a shared checkpoint on the same queries; subsequent repeated-update runs can then reveal how the different generators evolve. Judge the result by high-reward conditional performance in the real environment, used only for evaluation, rather than by model agreement or imagined reward alone.

4. Repeated sampling and the stopping property

Running consistency on the model’s own rollouts raises the question of feedback: an example is sampled, training changes the model, it is sampled again. For an

individual interval query $y = (h_i, x, r)$,

$$\nabla_{\theta} \mathcal{L}_{\text{cons}}(y) = 2\Delta_{\theta}(y)\nabla_{\theta}\Delta_{\theta}(y).$$

At a model that satisfies the identity on every sampled query,

$$\Delta_{\theta}(y) = 0 \implies \nabla_{\theta} \mathcal{L}_{\text{cons}}(y) = 0 \quad \text{for each } y.$$

Consequently its sampled gradient has zero variance there as well as zero mean. A repeatedly sampled, already-consistent trajectory receives no additional update merely because it appears again. There is no “sampled again, therefore imitate again” mechanism, because nothing is imitated.

This does not establish global stability. Away from agreement, oversampling some queries changes the updates; shared parameters couple queries; offline fitting continues to move the model; and a wrong or reward-ignoring model can also be consistent. Finite steps on squared log residuals can overshoot. Every head, including the reward head, remains trainable throughout.

Remark: relation to temporal-difference learning

Summed over continuations, the identity is the policy-evaluation Bellman equation for the NOR behavior, distributional over reward bins (section 1). The pointwise identity is stronger than a sampled Bellman residual: a squared residual on one sampled successor is nonzero at the true model because q_i is an expectation, whereas the conditioned prediction absorbs that fluctuation and $\Delta_{i,j}(r) = 0$ holds for every continuation. It also needs no importance correction under an off-policy proposal.

With full gradients the objective is gradient descent on a bounded residual, so it cannot diverge; its costs are slow propagation and the degenerate agreements above. A stop-gradient on the later reward endpoint, b_j , would turn the update into semi-gradient TD in the log domain: faster and directional from the terminal anchor, but off-policy bootstrapping with function approximation, with no convergence guarantee. It is well defined for the local one-step loss, not under the all-interval variance shortcut. It is not used; it is a possible ablation if propagation proves too slow.